

Bilingual ParaLang v26 Manual & Security Guide / Двомовний посібник та посібник із безпеки ParaLang v26

1. Introduction & Language Philosophy / Вступ та філософія мови

У сучасних високомасштабованих середовищах ефективна обробка паралелізму є критичною вимогою для системного та веб-програмування. Оскільки мережеві запити та дискові операції введення-виведення на порядки повільніші за обчислення, ParaLang v26 виступає стратегічним освітнім інструментом, що навчає розробників підтримувати безперервність обчислень, доки система очікує на зовнішні ресурси. Використовуючи досвід високонавантажених систем, ParaLang дозволяє студентам опанувати мистецтво розподілу навантаження між ядрами процесора, забезпечуючи максимальну пропускну здатність. In modern high-scalability environments, the effective handling of concurrency is a critical requirement for systems and web programming. Because network requests and disk IO are orders of magnitude slower than computation, ParaLang v26 serves as a strategic educational tool, teaching developers to maintain computational continuity while the system awaits external resources. Leveraging insights from high-load systems, ParaLang enables students to master the art of distributing workloads across CPU cores, ensuring maximum throughput.

1.1. Core Definition / Основне визначення

ParaLang — це мультипарадигмальна мова, розроблена для синтезу функціональної стійкості Elixir та системного прагматизму Go. Її навчальна місія полягає у наданні розробникам можливості вибору між високорівневими абстракціями (як-от відмовостійкі актори Erlang) та низькорівневим керуванням ресурсами (як-от компільований машинний код). Мова дозволяє зрозуміти, як різні підходи до паралелізму впливають на архітектуру сучасного ПЗ. ParaLang is a multi-paradigm language designed to synthesize the functional resilience of Elixir with the systems-level pragmatism of Go. Its educational mission is to provide developers with a choice between high-level abstractions (such as Erlang's fault-tolerant actors) and low-level resource management (such as compiled machine code). The language facilitates an understanding of how different concurrency approaches influence modern software architecture.

1.2. Supported Concurrency Paradigms / Підтримувані парадигми паралелізму

- **CSP (Communicating Sequential Processes):** Використовує анонімні горутини та адресовані канали для синхронізації.
- *Стратегічний вплив:* Дозволяє створювати слабкозв'язані компоненти, мінімізуючи накладні витрати на синхронізацію.
- **Actor Model (Модель акторів):** Базується на ізольованих процесах (PID), що обмінюються повідомленнями.
- *Стратегічний вплив:* Забезпечує безшовне горизонтальне масштабування через комірчасті мережі (mesh networks) та повну прозорість розташування процесів.
- **Pthreads (POSIX Threads):** Надає прямий контроль над потоками ОС та примітивами спільної пам'яті.

- *Стратегічний вплив:* Гарантує максимальну утилізацію ресурсів через пряме відображення на апаратні потоки.
- **STM (Software Transactional Memory):** Використовує TVar та атомарні блоки для керування станом.
- *Стратегічний вплив:* Повністю усуває ризики взаємоблокувань (deadlocks) у середовищах із високою конкуренцією за пам'ять.
- **OpenMP & MPI:** Паралелізм на рівні циклів та колективна комунікація між вузлами.
- *Стратегічний вплив:* Забезпечує ефективність наукових обчислень та обробку масивних даних на кластерах.
- **CUDA:** Розширення обчислень на GPU.
- *Стратегічний вплив:* Дозволяє досягати кратного прискорення обчислень (Nx) завдяки масивному паралелізму графічних ядер.
- **CSP (Communicating Sequential Processes):** Uses anonymous goroutines and addressable channels for synchronization.
- *Strategic Impact:* Allows for decoupled, anonymous task execution, reducing synchronization overhead.
- **Actor Model:** Based on isolated processes (PIDs) exchanging messages.
- *Strategic Impact:* Enables seamless horizontal scaling across mesh networks with full location transparency.
- **Pthreads (POSIX Threads):** Provides direct control over OS threads and shared memory primitives.
- *Strategic Impact:* Facilitates maximum resource utilization through direct hardware thread mapping.
- **STM (Software Transactional Memory):** Uses TVar and atomic blocks for state management.
- *Strategic Impact:* Ensures data consistency without the risk of deadlocks in high-contention memory environments.
- **OpenMP & MPI:** Cycle-level parallelism and collective communication across nodes.
- *Strategic Impact:* Leverages collective communication for scientific-grade massive data processing.
- **CUDA:** Computation offloading to GPU.
- *Strategic Impact:* Achieves magnitude-level speedups (Nx) by leveraging the massive parallelism of graphics cores.

1.3. Educational Objective / Навчальна мета

ParaLang прагне навчити розробників розрізняти низькорівневі механізми безпеки пам'яті (як у Rust) та високорівневі стратегії відмовостійкості (як у Elixir). Порівнюючи ці підходи, студенти вчаться знаходити баланс між чистою продуктивністю та надійністю розподілених систем. The objective of ParaLang is to teach developers to distinguish between low-level memory safety mechanisms (as in Rust) and high-level fault-tolerance strategies (as in Elixir). By contrasting these approaches, students learn to balance raw performance with the reliability of distributed systems.

2. Protected Execution & Security Guide / Захищене виконання та безпека

В академічному та професійному середовищі існує постійна напруга між відкритістю коду на платформах типу GitHub та необхідністю захисту інтелектуальної власності або академічної доброчесності. ParaLang v26 вирішує це через механізми захищеного виконання. In academic and professional settings, there is a constant tension between open-source distribution on platforms like GitHub and the need for intellectual property protection or academic integrity. ParaLang v26 addresses this through protected execution mechanisms.

2.1. Compilation via Nuitka & PyInstaller / Компіляція через Nuitka та PyInstaller

ParaLang трансліює високорівневий код у виконувані файли C++. Це не лише захищає логіку від реверс-інжинірингу, а й забезпечує стратегічну перевагу в ресурсах. Спираючись на галузеві бенчмарки, ParaLang демонструє ефективність: використання пам'яті становить лише 50 МБ (подібно до Rust), що значно перевершує показники Elixir (295 МБ) та Go (310 МБ), зберігаючи при цьому високу швидкість виконання. ParaLang translates high-level code into C++ executables. This not only protects logic from reverse engineering but also provides a strategic resource advantage. Based on industry benchmarks, ParaLang demonstrates superior efficiency: memory footprint is only 50MB (matching Rust), significantly outperforming Elixir (295MB) and Go (310MB), while maintaining high execution speed.

2.2. Bytecode Obfuscation (.pyc) / Обфускація байт-коду (.pyc)

Попередня компіляція в байт-код дозволяє запускати ПЗ без надання вихідних файлів. **Security vs. Accessibility / Безпека проти доступності** Використання .pyc приховує алгоритмічну логіку від поверхневого аналізу, дозволяючи студентам використовувати складні модулі як "чорні скриньки" без ризику плагіату. The use of .pyc hides algorithmic logic from superficial analysis, allowing students to use complex modules as "black boxes" without the risk of plagiarism.

3. Detailed Language Syntax Guide / Детальний опис синтаксису

Спеціалізований синтаксис ParaLang розроблений для чіткого управління станами та запобігання побічним ефектам у паралельних задачах. The specialized syntax of ParaLang is designed for precise state management and the prevention of side effects in concurrent tasks.

3.1. CSP & Actor Model (Channels & PIDs) / CSP та модель акторів

ParaLang розмежовує "адресовані канали" (CSP) та "адресовані актори" (Actor Model).

- У моделі CSP горутини залишаються анонімними, а канали — основним об'єктом взаємодії.
- У моделі акторів кожен процес отримує унікальний PID. Завдяки прозорості мережі, актори можуть обмінюватися повідомленнями між різними вузлами кластера, що робить перехід від моноліту до мікросервісів мінімальним. ParaLang distinguishes between "addressable channels" (CSP) and "addressable actors" (Actor Model).
- In CSP, goroutines remain anonymous, and channels are the primary interaction objects.

- In the Actor model, every process receives a unique PID. Due to location transparency, actors can exchange messages across different cluster nodes, making the transition from monolith to microservices minimal.

3.2. POSIX Threads (Pthreads) & Synchronization / POSIX Threads та синхронізація

ParaLang використовує асинхронні примітиви синхронізації (Mutex, RwLock), засновані на архітектурі Tokio.

- **Важливо:** Ці блокування є неблокуючими для потоків (non-blocking for threads). Вони зупиняють лише конкретну задачу (task), дозволяючи іншим задачам продовжувати роботу на тому ж потоці ОС. Це запобігає простою ядра процесора. ParaLang utilizes asynchronous synchronization primitives (Mutex, RwLock) based on the Tokio architecture.
- **Critical Detail:** These locks are non-blocking for threads. They only suspend the specific task, allowing other tasks to continue execution on the same OS thread, preventing CPU core idleness.

3.3. Software Transactional Memory (STM) / Програмна транзакційна пам'ять

STM реалізує концепцію atomically блоків. Використовуючи retry, система автоматично відстежує зміни у TVar. Якщо транзакція не може бути завершена через конфлікт, вона переривається без блокування всієї системи та автоматично перезапускається, коли дані змінюються. STM implements the concept of atomically blocks. Using retry, the system automatically tracks changes in TVar. If a transaction cannot be completed due to a conflict, it aborts without blocking the entire system and automatically retries once data changes.

4. The Quantum Multiverse Multiblock (v26) / Квантовий мультиблок multiverse(N)

Виявлення станів гонитви (race conditions) у традиційних мовах ускладнене через кооперативну багатозадачність. Наприклад, у Go контекст перемикається лише під час комунікації, що дозволяє "щільним циклам" (tight loops) захоплювати ядра. Detecting race conditions in traditional languages is complicated by cooperative multitasking. For instance, Go only switches context during communication, allowing "tight loops" to hog cores.

4.1. Concept of Randomized Schedules / Концепція рандомізованих розкладів

Блок multiverse(N) виконує код у N "паралельних всесвітах" одночасно. Кожна ітерація має унікальний рандомізований розклад перемикавання контексту, що дозволяє виявити помилки, які виникають лише за специфічних таймінгів. The multiverse(N) block executes code across N "parallel universes" simultaneously. Each iteration features a unique randomized context-switching schedule, exposing bugs that only surface under specific timings.

4.2. Automatic Data-Race Detection / Автоматичне виявлення станів гонитви

На відміну від Go чи Haskell, ParaLang реалізує повну витісняльну багатозадачність із "бюджетом редукцій" (2000 операцій). Це гарантує, що жодна задача не зможе монополізувати ядро. Система моніторингу фіксує будь-який недетермінізм у стані спільних змінних, сигналізуючи про наявність стану гонитви. Unlike Go or Haskell, ParaLang implements full preemptive multitasking with a "reduction budget" (2000

operations). This ensures no task can monopolize a core. The monitoring system captures any non-determinism in shared variable states, flagging the presence of race conditions.

5. Annotated Code Examples / Приклади коду з коментарями

5.1. OpenMP: Monte Carlo Pi Calculation / Розрахунок Пі

```
// Розподіл ітерацій між ядрами / Iteration distribution
pfor (i = 0; i < 10^7; i++) {
    if (in_circle(rand())) {
        // Атомарне оновлення / Atomic update
        atomic_increment(counter);
    }
}
```

5.2. CSP: Concurrent Crawler / CSP: Паралельний краулер

```
ch = make(chan url)
spawn(fn {
    links = extract_links(url)
    ch <- links // Передача через канал / Send via channel
})
// Отримання (Go-style) / Receive
result = <-ch
```

5.3. Multiverse: Catching a Race Condition / Виявлення гонитви

```
multiverse(100) {
    count = 0
    // Без Mutex виникне гонитва / Race without Mutex
    spawn(fn { count += 1 })
    spawn(fn { count += 1 })
    // multiverse(100) покаже різні результати / will show varying
    results
}
```

ParaLang v26 є надійним містком між академічною теорією та професійною системною інженерією, забезпечуючи розробників інструментами для створення відмовостійкого ПЗ майбутнього. ParaLang v26 stands as a robust bridge between academic theory and professional systems engineering, providing developers with the tools to create the fault-tolerant software of the future.